

# RBS 系统 · Java 微服务架构设计说明

技术栈：Java 17 · Spring Boot 3.5 · Spring Cloud Alibaba · Nacos · Gateway · PostgreSQL · Redis · RabbitMQ · Flowable  
运行环境：阿里云（ECS · Docker 编排 · 托管中间件）· 技术基座：平台基座 · 业务领域：销售与服务全链路七大域

本文档描述 RBS 系统 **Java 微服务** 目标架构：整体建立在**阿里云基础设施**之上；平台基座与 RBS 业务域协同，业务按客户、商机、销售、交付、风控、财经、保险等七大业务域拆分，支持单体聚合与独立部署。

## 升级前背景（现状痛点）

现行系统在向领域化微服务演进过程中，需优先治理以下结构性问题（亦是本册各章节设计的出发点）：

- **服务边界混乱**：微服务运行进程多达 25+，按功能模块而非业务领域划分，域间职责交叉严重，跨模块调用普遍，难以独立演进与扩容。
- **代码与仓库分散**：项目代码散落在 70+ 个 Git 仓库，缺少统一工程视图；开发人员难以识别系统全貌，无法高效跟踪跨仓库变更与版本关系。
- **重复代码泛滥**：获取用户、查询客户等基础能力在各服务重复实现，行为不一致、维护成本高。
- **单库表模式混乱**：所有数据挤在一个数据库中，库内表与模式（schema）边界不清，**配置数据、运行态数据、业务落盘数据**混杂，难以辨识归属与变更影响，维护与排障成本高。
- **存储过程过重**：大量复杂存储过程长时间占用数据库连接与锁资源，性能与可观测性差，应用层难以治理。
- **任务调度混用**：XXL-Job 与 @Scheduled 等调度注解并存且缺乏统一治理，任务执行不可追踪、不可控、难扩容。
- **工程流程缺失**：缺少独立开发/测试环境与规范流程；开发人员常直连**线上环境**（含正式库）联调，并依赖**手动注释代码**（如定时任务、MQ 消费、审批回调等）规避副作用，易漏提交、难复现，质量与数据安全风险高。

## 系统定位

面向工程机械/设备 **销售与服务全链路**：获客 → 赢单 → 成交 → 交付。平台能力由基座统一提供；业务域按领域模型拆分为独立 Maven 模块与微服务，支持单体聚合（platform-server）与独立部署（各业务域服务）两种形态。

## 架构范围

- **业务域** — 入口/门户、核心链路（获客→交付）、支撑域（风控/财经/保险）、外部集成；OA·HRM 等遗留模块并行（见第12章）
- **数据架构** — cfg / data / log / stats 分库 · data 库 schema 分域（第3章）
- **工程视图** — 总体/部署/治理/异步任务/分层/安全/缓存/CI/CD/可观测性
- **异步任务** — Job 定义中心、API / XXL-Job 双通道发布、Worker 并行与单实例

- 平台基座 — 提供 gateway、system（含 infra）、bpm、framework 等平台能力
- 分析与报表 — DataEase BI（只读副本），不纳入 Java 微服务进程
- 阿里云基础设施 — ECS/SLB/CDN · RDS PostgreSQL · 云 Redis · 消息队列 · OSS · 日志与监控旁路

设计目标（摘要）

- 边界清晰：一域一服务；跨域仅 \*-api / Export 契约，禁止依赖他域 server
- 可水平扩展：无状态 API + Gateway 负载；MQ 事件解耦；Job Worker 池横向扩容
- 任务统一：Job 定义集中治理；API 与 XXL-Job 共用 jobCode；Worker 支持并行 / 单实例
- 可演进：分阶段上线业务域；基座保持稳定
- 安全统一：企微 SSO → JWT；公司级 / BU 级数据权限
- 能力复用：用户/客户/组织等由基座或归属域 唯一实现，他域仅依赖 Export / \*-api
- 数据分库分域：cfg / data / log / stats 四库职责分离；data 库内以 schema 对应业务领域（见第3章）
- 数据下沉：复杂存储过程分阶段下线，逻辑迁至应用层与 Job；报表走 BI 只读副本
- 调度红线：业务服务禁止 @Scheduled；任务必须登记 jobCode，经 API 或 XXL-Job 发布
- 工程规范：dev / test / pre / prod 环境隔离，禁止非生产连接线上库；用配置/开关替代手注释代码规避任务与消费
- 工程收敛：70+ 仓库向 RBS Monorepo + 域模块归集；25+ 进程收敛至约 12 个可运维单元（见第6章）
- 云上部署：生产运行于阿里云（ECS · Docker Compose · SLB/CDN · RDS · 云 Redis · OSS）

升级对策对照（痛点 → 目标）

| 升级前痛点         | 升级后对策（本册）                                               |
|---------------|---------------------------------------------------------|
| 25+ 进程、边界乱    | 七大域 + 一域一服务；第6章进程全景；跨域 Export（第4章）                      |
| 70+ 仓库、难掌握全貌  | RBS Monorepo + 域模块；第11章统一构建发布；本册章节索引                    |
| 用户/客户重复实现     | Export 契约 + system/客户域唯一实现（第4章）                         |
| 单库表模式混乱       | cfg/data/log/stats 四库 + data 库按域 schema；配置/运行/落盘分责（第3章） |
| 存储过程过重        | 逻辑下沉应用层/Job；PG + 分阶段下线 SP；BI 只读（第2章）                    |
| XXL-Job 与注解混用 | Job 定义中心 + jobCode；禁用业务 @Scheduled（第5章）                 |
| 连线上环境、手注释代码   | dev/test 独立环境；禁止连生产；配置/特性开关替代注释；第11章门禁                  |

业务与协作

- 1. 总体架构图
- 3. 数据架构图
- 5. 异步任务与 Job Worker

工程与运维

- 6. 微服务治理架构
- 8. 部署架构图
- 10. 缓存架构图
- 12. 可观测性架构

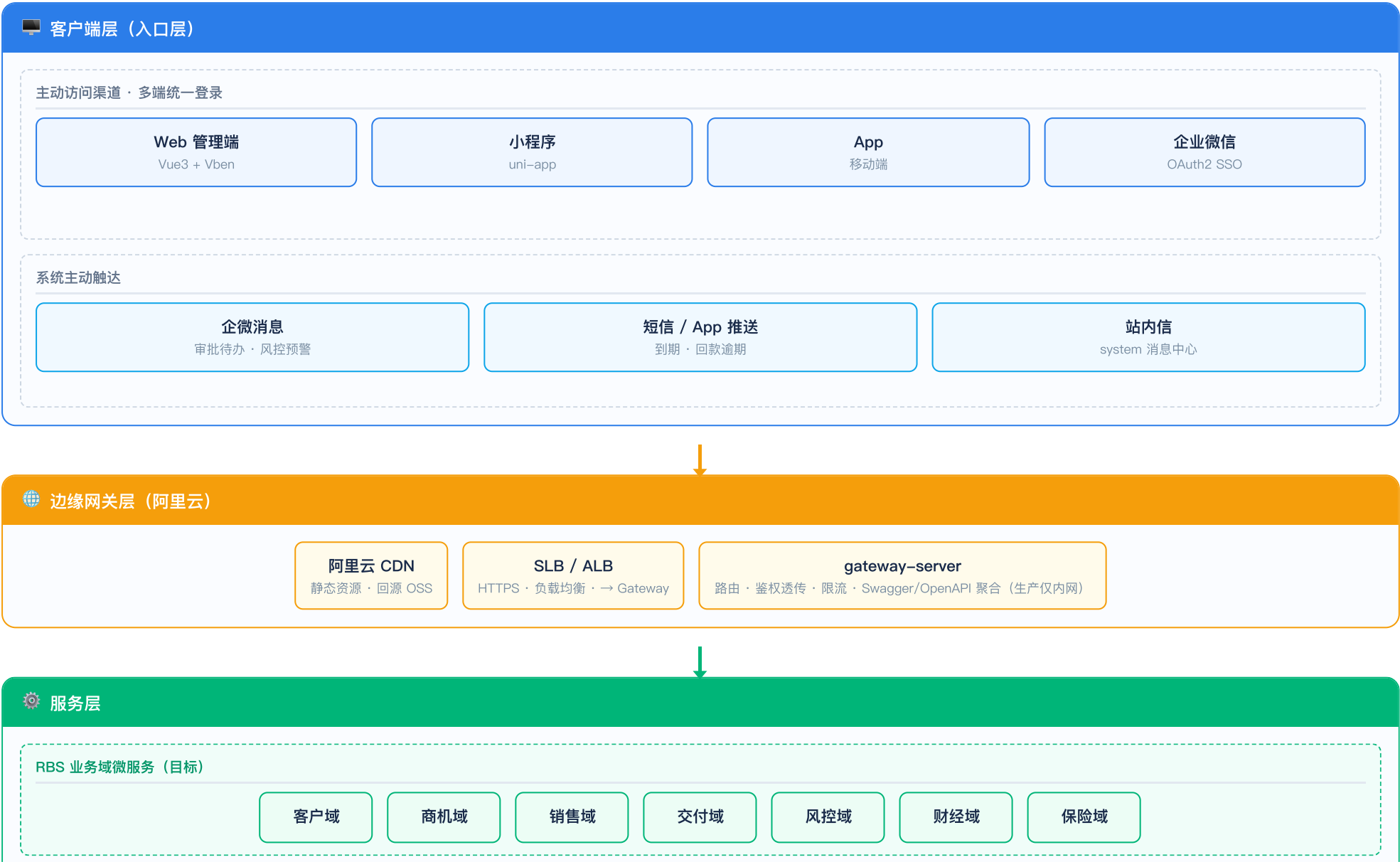
- 2. 业务领域架构图
- 4. 跨域协作与 Export API

- 7. 分层架构图
- 9. 安全架构图
- 11. CI/CD 发布流程

# RBS 系统 · 总体架构图

客户端 → 阿里云边缘 → Gateway → RBS 业务域 + 平台基座 → 托管中间件与数据层

设计意图：自上而下展示访问路径；服务层拆为 平台基座 与 RBS 业务域 两条泳道。





**运行底座：**阿里云 ECS·Docker 编排·托管中间件（RDS·云 Redis·消息队列·OSS）

**数据架构：**cfg / data / log / stats 四库分责；data 库内 schema 标识业务领域（详见第3章）。DataEase 连只读副本。SP 分阶段下线。

**数据演进：**复杂逻辑迁至应用层/Job；业务写主库，BI 只读副本

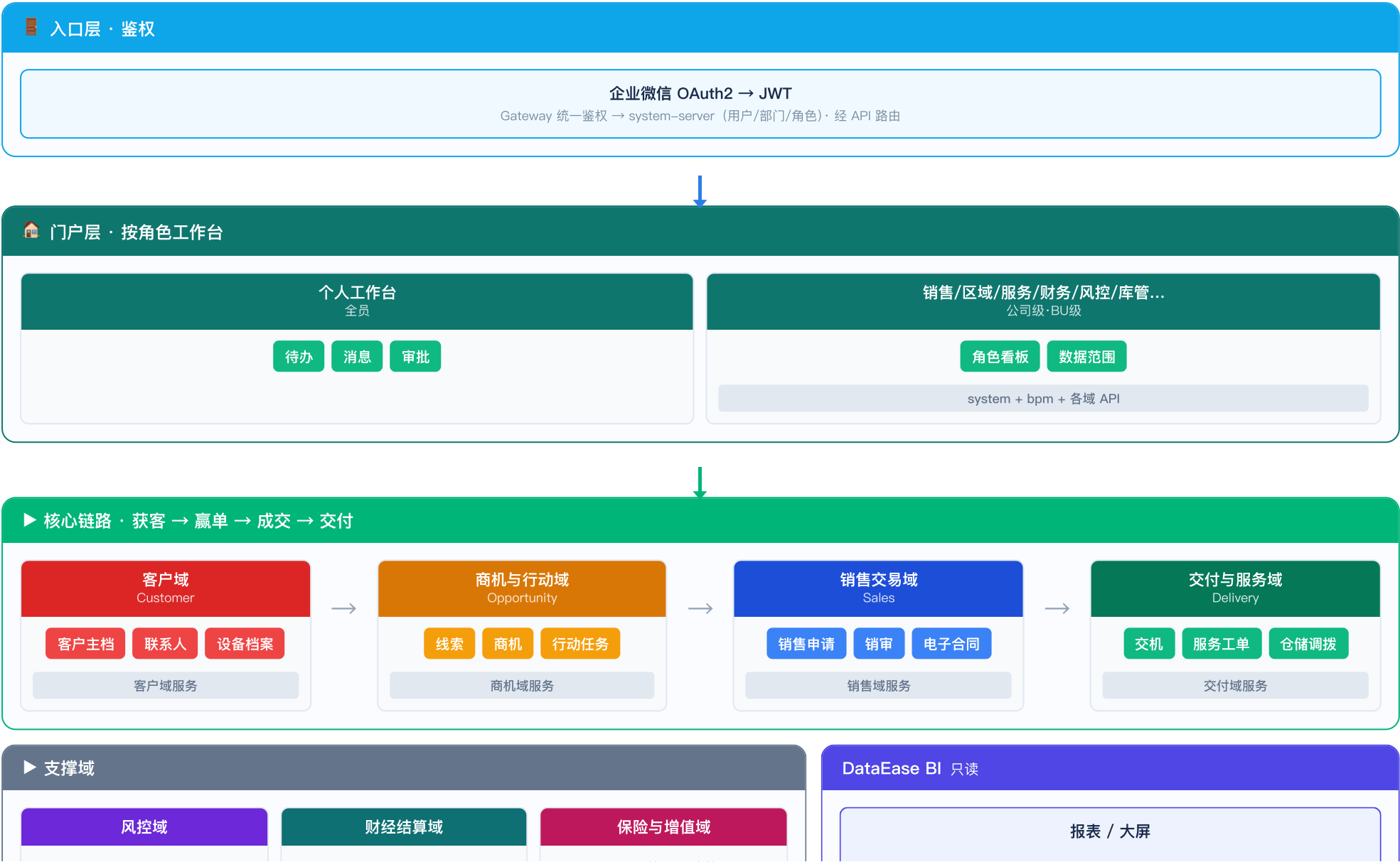
**企业集成（非业务域）：**rbs-integration-\* 防腐层/适配器·CRM/ERP/企微/支付/物联网·同步数据落各域 schema·大批量走 Job Worker（详见第3章、第6章）

**横切能力（贯穿各层）：**Spring Security + OAuth2/JWT·多租户/公司/BU 数据权限·操作日志·统一异常·MDC 请求标识·统一 Job 发布·三级缓存

# RBS 系统 · 业务领域架构图

入口/门户、核心链路、支撑域、平台与基础设施

设计意图：自上而下为入口 → 门户 → 业务域 → 基础支撑 → 基础设施。





## RBS 系统 · 数据架构图

cfg / data / log / stats 四库分责 · data 库内 schema 标识业务领域 · BI 只读副本

设计意图：配置、运行、落盘、统计分库存放；业务数据集中在 data 库，以 schema 对应领域，解决单库混表维护难题。

## 升级前 · 数据层痛点

- 全部对象落在同一数据库，表与 schema 缺乏统一规则。
- 配置（字典/参数/Job 元数据）、运行态（会话/锁/临时态）、业务落盘混杂，变更易误伤、难排障。
- 无法从库结构判断表归属哪个业务域，跨团队维护成本高。

## 四库分责 (RDS PostgreSQL · 逻辑 database)



 rbs\_data · schema 标识业务领域

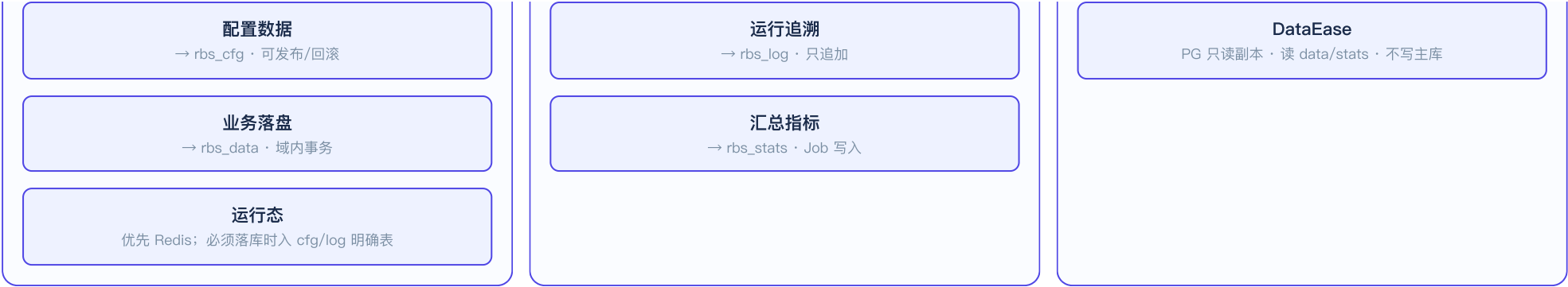


### 三类数据归位

## 日志与统计

只读分析





**环境:** dev/test/pre/prod 各维护四套逻辑库; 禁止非生产连生产库 (第11章 CI/CD)。  
**演进:** 存量单库 → 先划 schema → 再拆四库 → 远期可按域独立 RDS。  
**关联:** 存储过程分域下线见第2章; 跨域协作见第4章; 异步汇总见第5章。

# RBS 系统 · 跨域协作与 Export API

纯 Java 契约层（非 Feign Client）；微服务下可选 Feign Adapter

设计意图：打破 api 循环依赖；业务代码只依赖 Export 接口，不依赖他域 server 实现。



Export 契约（推荐）

**rbs-export-user.jar**  
UserExportService 接口 + DTO · 无 @FeignClient

**system-server 实现**  
UserExportServiceImpl

**销售域服务 依赖**  
仅 export-user · 不依赖 system-server

首期 Export: UserExport · CustomerExport · DeptExport · DictExport（基座/system 实现，他域只引 jar）

协作方式优先级

1. Export API — 编译期契约 · 单体本地 Bean

2. 平台 \*-api + Feign — 基座已有（AdminUserApi 等）

3. RabbitMQ 领域事件 — 解耦通知

4. Job 异步任务 — API / XXL-Job · Worker（第5章）

5. BPM 回调 — FlowBillService 模式（OA 已实践）

不做 跨服务 Seata 2PC；跨域写采用 Saga / 补偿 / 幂等回调；MQ 发布建议 Outbox 模式。

单据 + 流程协作示例（销售申请）

销售域 落库 → BpmProcessInstanceApi 发起流程 → bpm 审批结束 → Feign/MQ → 销售域 更新 process\_status

端到端路径

提交

保存销售单 → submitProcessInstance → 回写 processInstanceld

审批

bpm-server · 工作流审批 → Flowable → 状态事件

回调

统一流程回调 (businessKey 幂等)



updateProcessStatus(businessKey)

# RBS 系统 · 异步任务与 Job Worker 架构

Job 定义统一管理 · API / XXL-Job 双通道发布 · Worker 并行扩展与单实例

设计意图：横切任务执行平台，与 RabbitMQ 领域事件互补；业务核心仍是领域边界与单据流程（第2章、第4章）。



多副本竞争消费 · 导出/同步

replicas: N

对账 · 日终 · 全局清理

replicas: 1 / SAC / 分布式锁

### 端到端示例 · 导出 vs 日终对账

发布源不同 · 同一 jobCode 模型 · Worker 按并发策略分流

API

POST .../export

→

JobPublisher.submit

→

MQ 队列

→

Worker xN

调度

XXL-Job Cron

→

jobCode 日终对账

→

同一通道

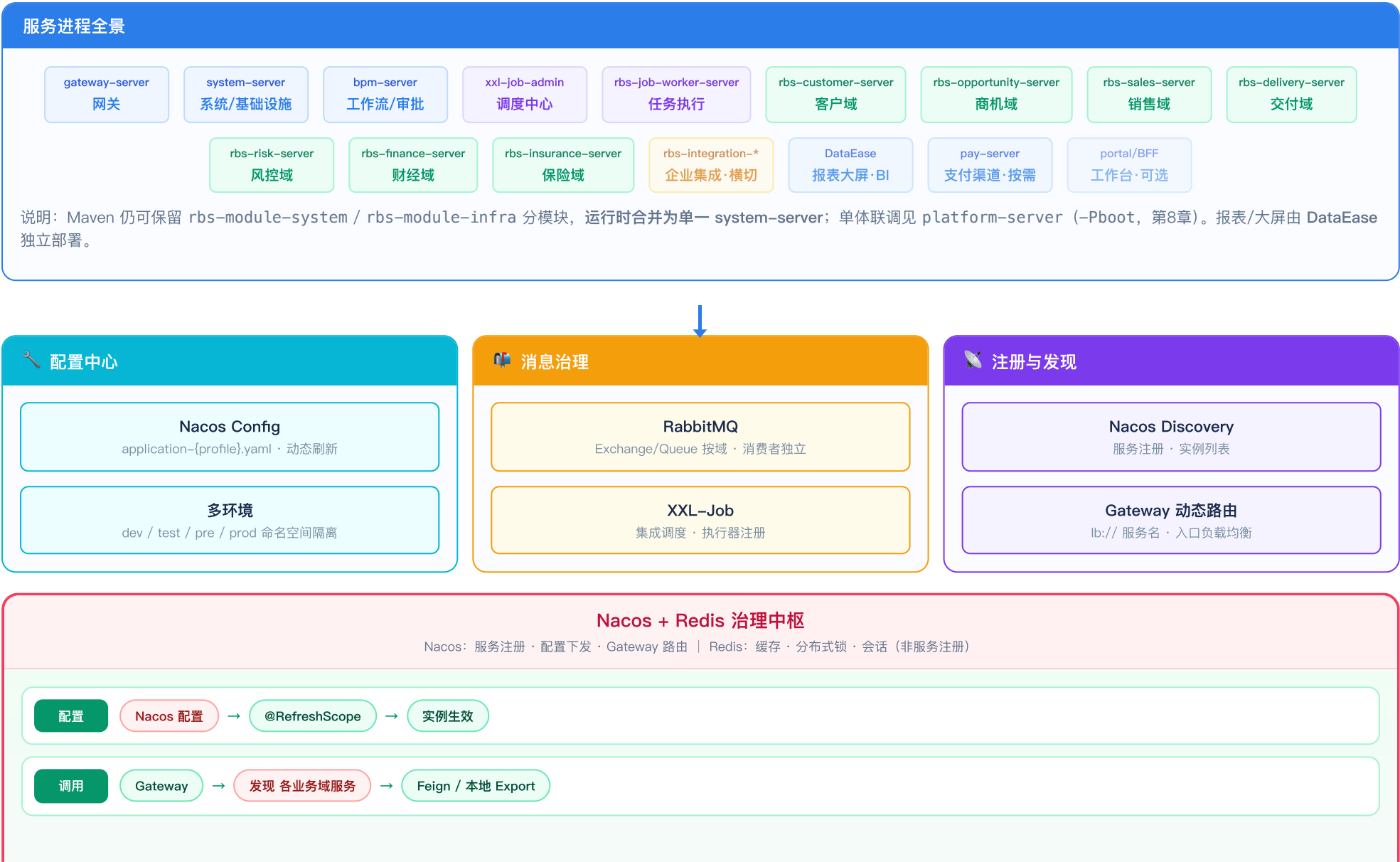
→

Worker 单实例

# RBS 系统 · 微服务治理架构

Nacos 为配置与注册发现枢纽

设计意图：配置 · 注册发现 · 消息 三列治理能力，与 Java 云原生栈对齐。



单体

platform-server



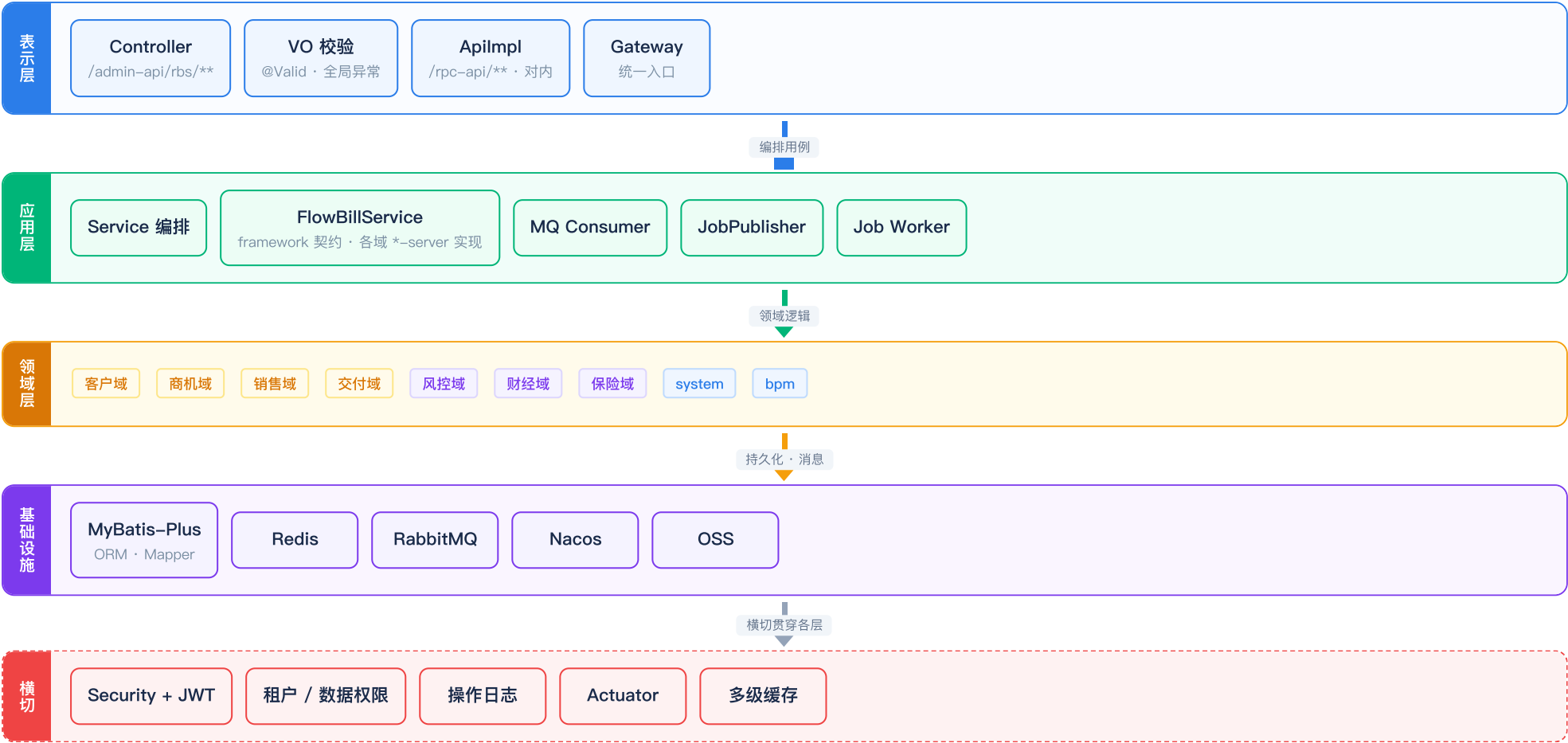
-Pboot 瘦 jar 聚合



Export 本地实现

# RBS 系统 · 分层架构图

表示层 → 应用层 → 领域层 → 基础设施 · 横切安全与可观测





# RBS 系统 · 部署架构图

阿里云 CDN/SLB → ECS · Docker Compose (Gateway · 平台 · BPM · Worker · 业务域) → 托管中间件 · ELK 旁路



按域独立容器 · compose 扩容

#### 阿里云托管中间件

**RDS · rbs\_cfg**  
配置库

**RDS · rbs\_data**  
业务库 · schema 分域

**RDS · rbs\_log**  
日志库

**RDS · rbs\_stats**  
统计库

云 Redis

Nacos (自建或 MSE)

RabbitMQ

#### 企业集成 (横切 · 非业务域)

**rbs-integration-\***  
CRM/ERP/企微/支付/物联网 · 适配器 jar

**同步 / 对账 Job**  
rbs-job-worker · 非独立业务库

#### 分析与 BI (非 Java 微服务)

**DataEase**  
报表/大屏 · PG 只读副本 · 独立容器

### 运维旁路

**ELK**  
日志汇聚

**Prometheus**  
Micrometer 指标

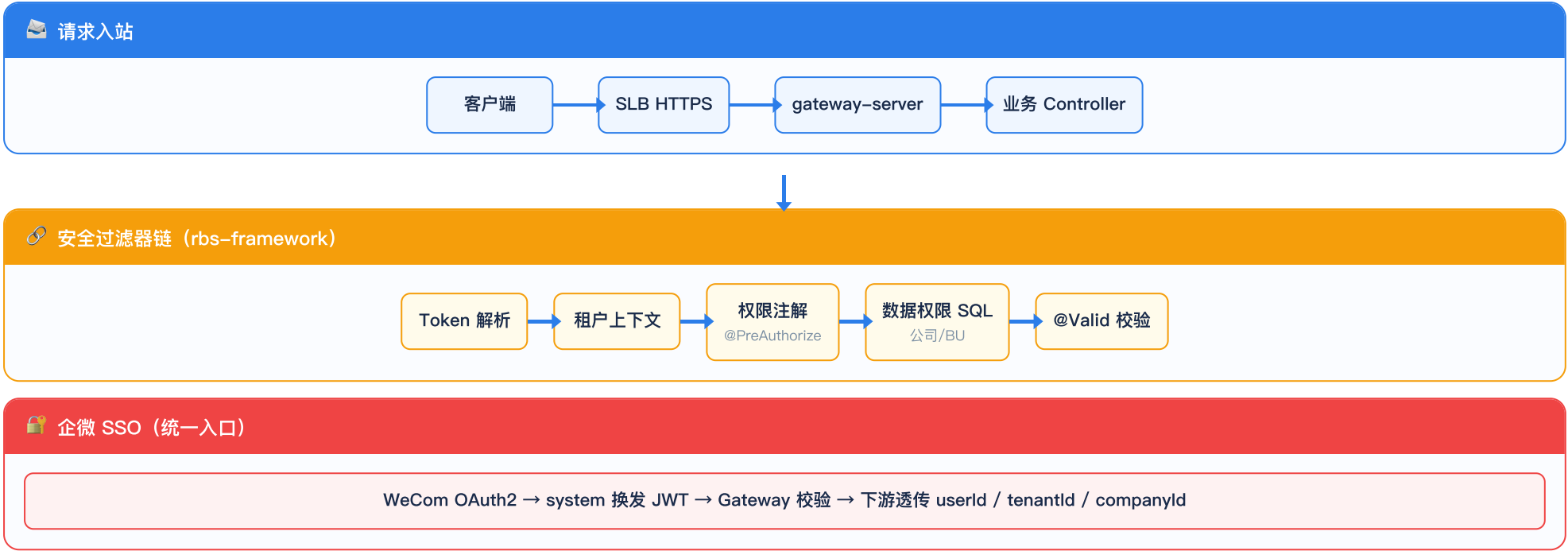
**Actuator**  
健康检查 · 端点

**开发环境:** mvn package -Pboot → java -jar platform-server.jar 单体联调; 亦可本地 docker compose 起依赖与中台服务。

**生产:** 阿里云 ECS 宿主机 · docker compose 编排 (非 ACK/K8s) · Pcloud 构建镜像 · 各服务独立容器 (含 bpm、job-worker) · Nacos (自建或 MSE) 注册 · 基线参考 platform-server/docker-compose.yml。

# RBS 系统 · 安全架构图

Gateway 统一鉴权 · Spring Security · 公司/BU 数据权限



# RBS 系统 · 缓存架构图

三级缓存 · 主页数据缓存 · 写路径以 DB 为准



# RBS 系统 · CI/CD 自动化发布流程

Git → Maven 构建 → Docker 镜像 → ECS Compose 部署 → 健康检查



| 环境矩阵（升级后强制）                                    |
|------------------------------------------------|
| <div>dev / test<br/>独立库 · 脱敏 · 禁止连生产 RDS</div> |
| <div>pre<br/>预发库 · compose 验证 · 审批后上生产</div>   |
| <div>prod<br/>生产 ECS · 仅 CI 发布 · 回滚预案</div>    |

| 后端发布要点                                                                               |
|--------------------------------------------------------------------------------------|
| <div>每域独立 Docker 镜像<br/>销售域服务.jar · ENTRYPOINT java -jar · Compose 一服务一容器</div>      |
| <div>平台与业务分批次发布<br/>先 system（含 infra）/bpm · 再 RBS 业务域服务</div>                        |
| <div>ECS · Docker Compose<br/>流水线/SSH 至 ECS · compose pull &amp; up -d · 非 K8s</div> |
| <div>Actuator /health 探活</div>                                                       |

# RBS 系统 · 可观测性架构图

日志 · 指标 · 业务预警

